



10-423/10-623 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Generative Adversarial Networks (GANs) + Probabilistic Graphical Models

Aran Nayebi & Matt Gormley

Lecture 6

Feb. 2, 2026

Reminders

- Homework 1: Generative Models of Text
 - Out: Mon, Jan 26
 - Slot A Due: Mon, Feb 9 at 11:59pm
- Remember, no AI use is permitted in Slot A! It will be considered an academic integrity violation. We want you to have the opportunity to think for yourself, so save the AI use for Slot B, when it's permitted!
- Regrade requests will only be for Slot B submissions, not Slot A (and only after Slot B is graded).
- If you got 100% on a question, you do not need to resubmit for Slot B (your score is the maximum of the two). In general, you do not need to submit anything to Slot B if you are happy with your Slot A grade!

TASK: IMAGE GENERATION

Image Generation

- Class-conditional generation
- Super resolution
- Image Editing
- Style transfer
- Text-to-image (TTI) generation



Figure from Razavi et al. (2019)

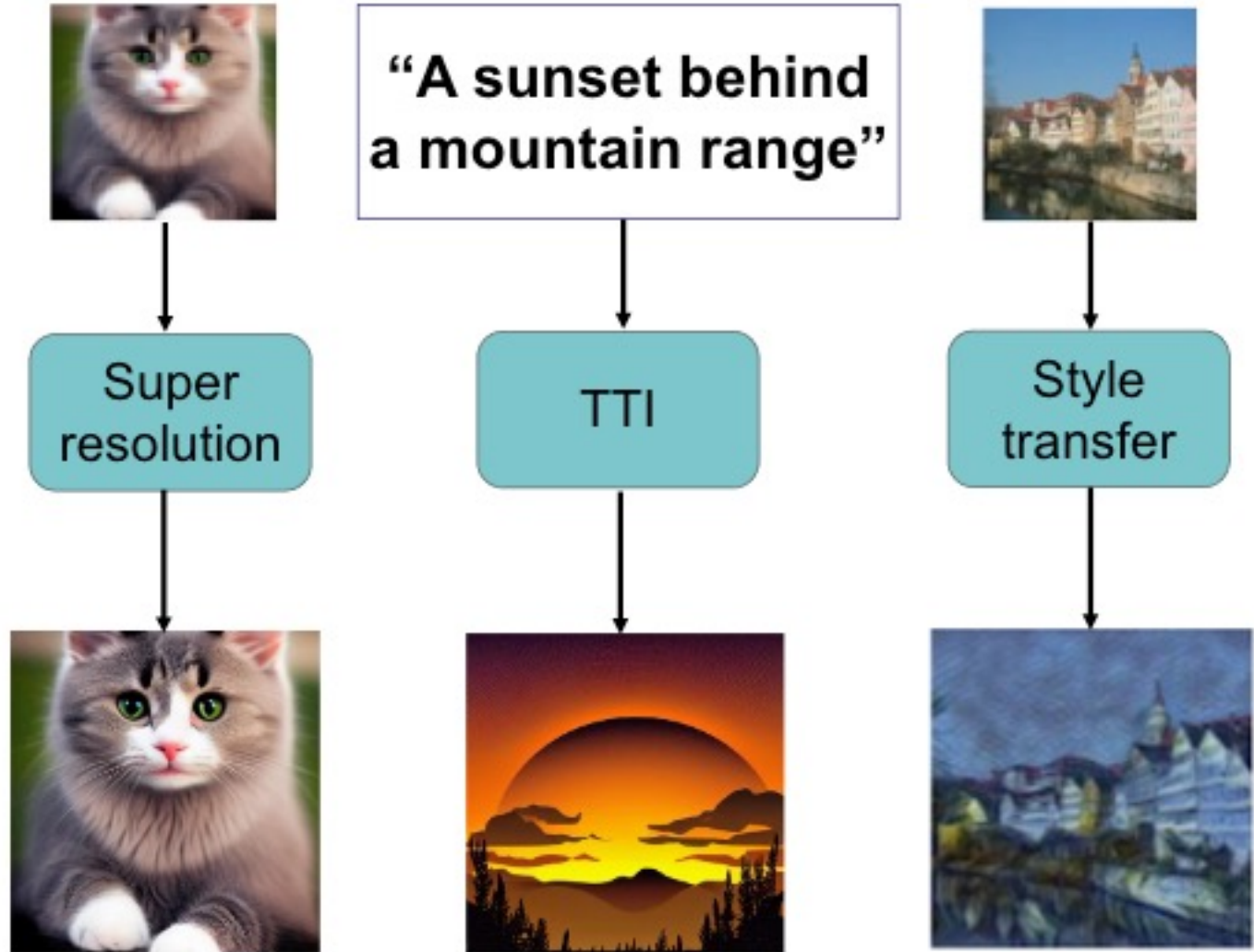


Figure from Bie et al. (2023)

Class Conditional Generation

- **Task:** Given a class label indicating the image type, sample a new image from the model with that type
- Image classification is the problem of taking in an image and predicting its label $p(y|x)$
- Class conditional generation is doing this in reverse $p(x|y)$

sea anemone

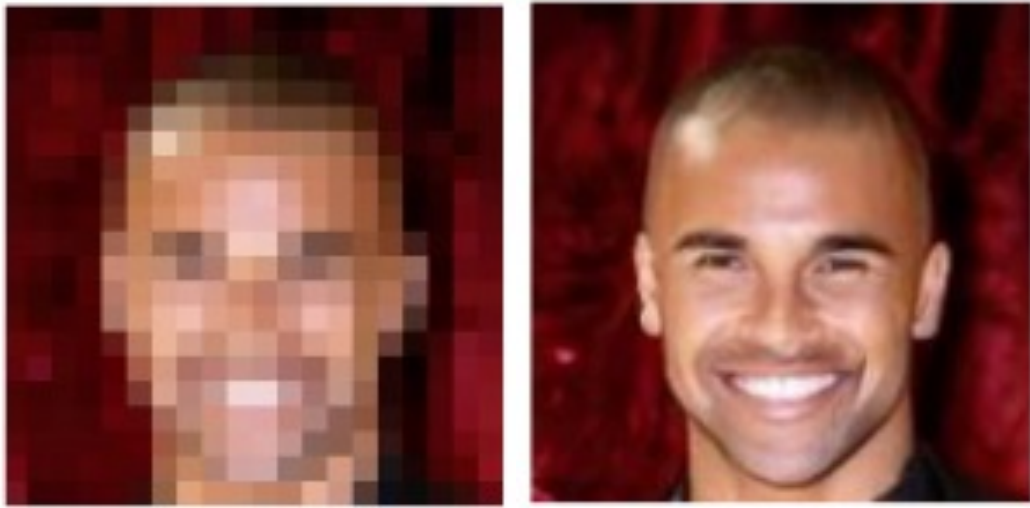
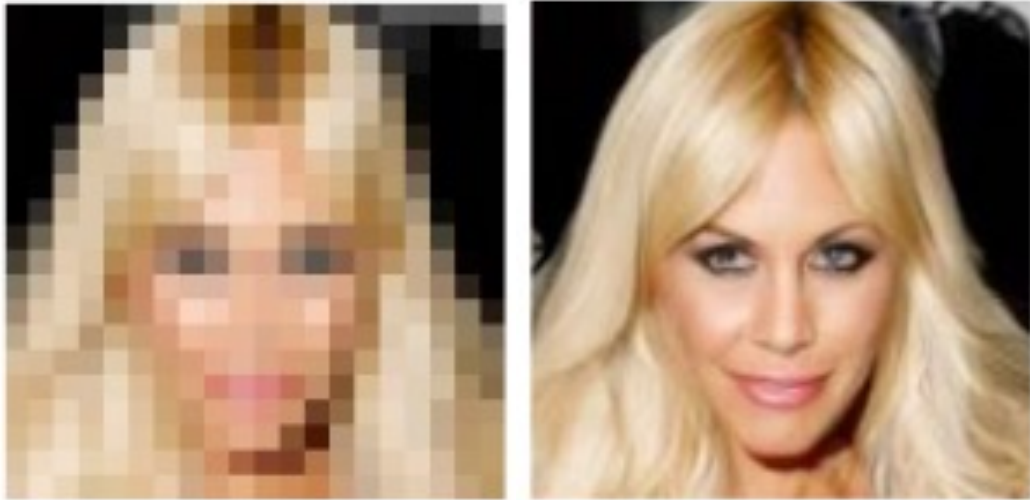
brain coral

slug

goldfinch



Super Resolution



LR

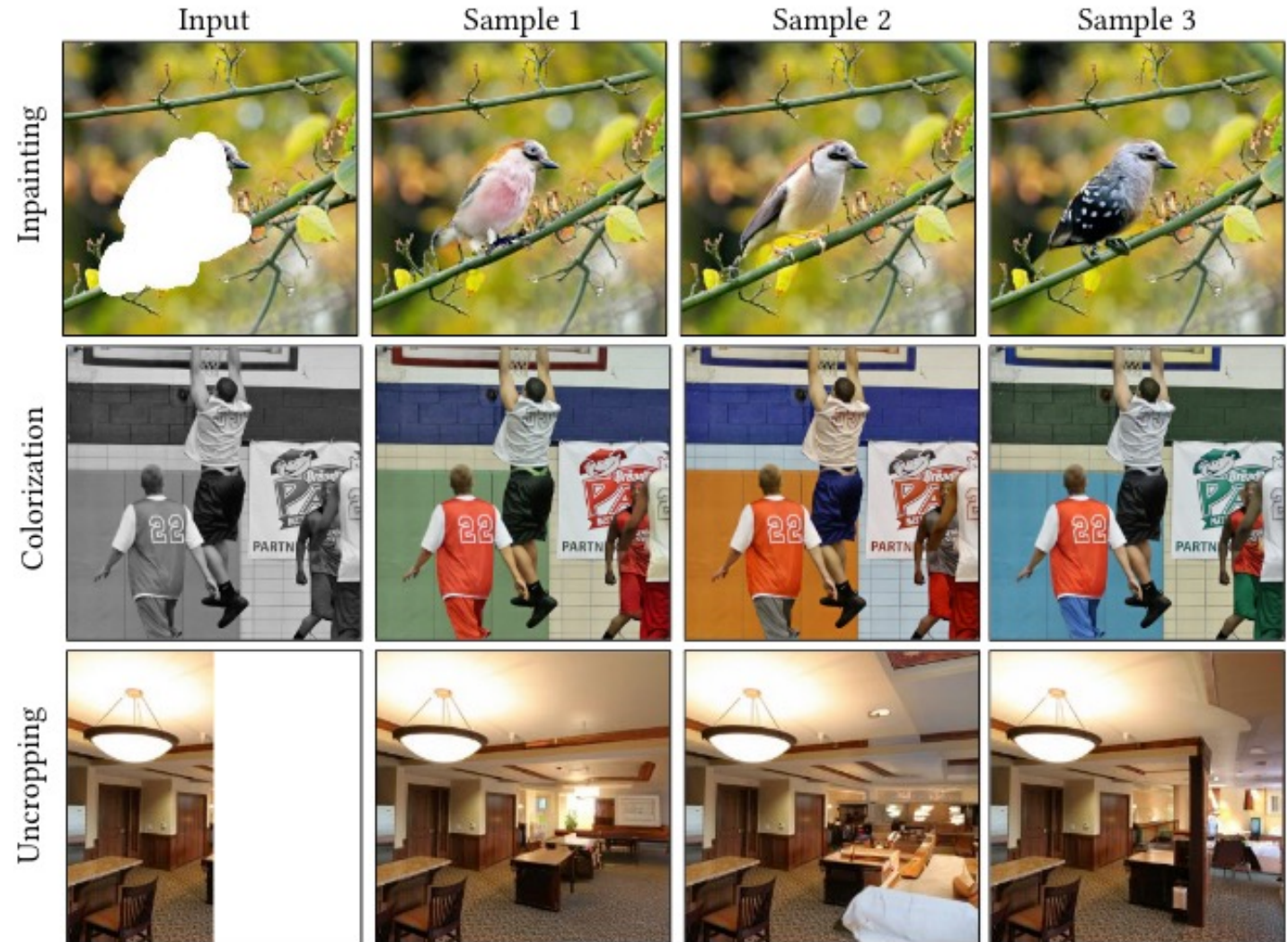
SRDiff

- Given a low resolution image, generate a high resolution reconstruction of the image
- Compelling on low resolution inputs (see example to the left) but also effective on high resolution inputs

Image Editing

A variety of tasks involve automatic editing of an image:

- **Inpainting** fills in the (pre-specified) missing pixels
- **Colorization** restores color to a greyscale image
- **Uncropping** creates a photo-realistic reconstruction of a missing side of an image



Style Transfer

- The goal of style transfer is to blend two images
- Yet, the blend should retain the semantic content of the source image presented in the style of another image



Figure from Gatys et al. (2016)

Text-to-Image Generation

- Given a text description, sample an image that depicts the prompt
- From Google Gemini 2.5 Flash Image (Nano Banana)

Figure from: https://www.linkedin.com/posts/alka-gupta-ag_so-i-bumped-into-taylor-swift-at-changi-airport-activity-7369534745061322753-296P



Text-to-Image Generation

- Given a text description, sample an image that depicts the prompt
- The following images are samples from SDXL with refinement

Prompt: close up headshot, futuristic **old man**, wild hair sly smile in front of gigantic UFO, dslr, sharp focus, dynamic composition, **rule of thirds**



MODEL: GENERATIVE ADVERSARIAL NETWORK (GAN)

Stable Diffusion (Spring 2026) still can't explain GANs

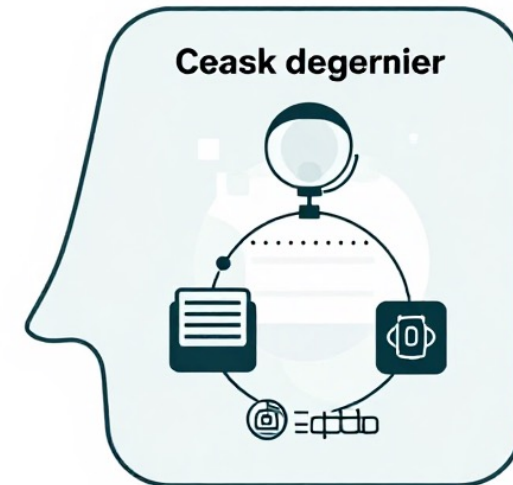
Prompt: slide explaining
Generative Adversarial
Networks (GANs) for a
Machine Learning course,
carefully designed, easy to
follow

Negative Prompt: boring,
unclear, nontechnical

Generative Adversarial Networks (GANs)

**GANs): tie inderict. and
peopicat maic encpem** to
Generative Adversarial knows
nabecal: that induss the a
cenracies in cacalion.

Aneido:
Emechpic ancogel
(Ncuption. The our trie).



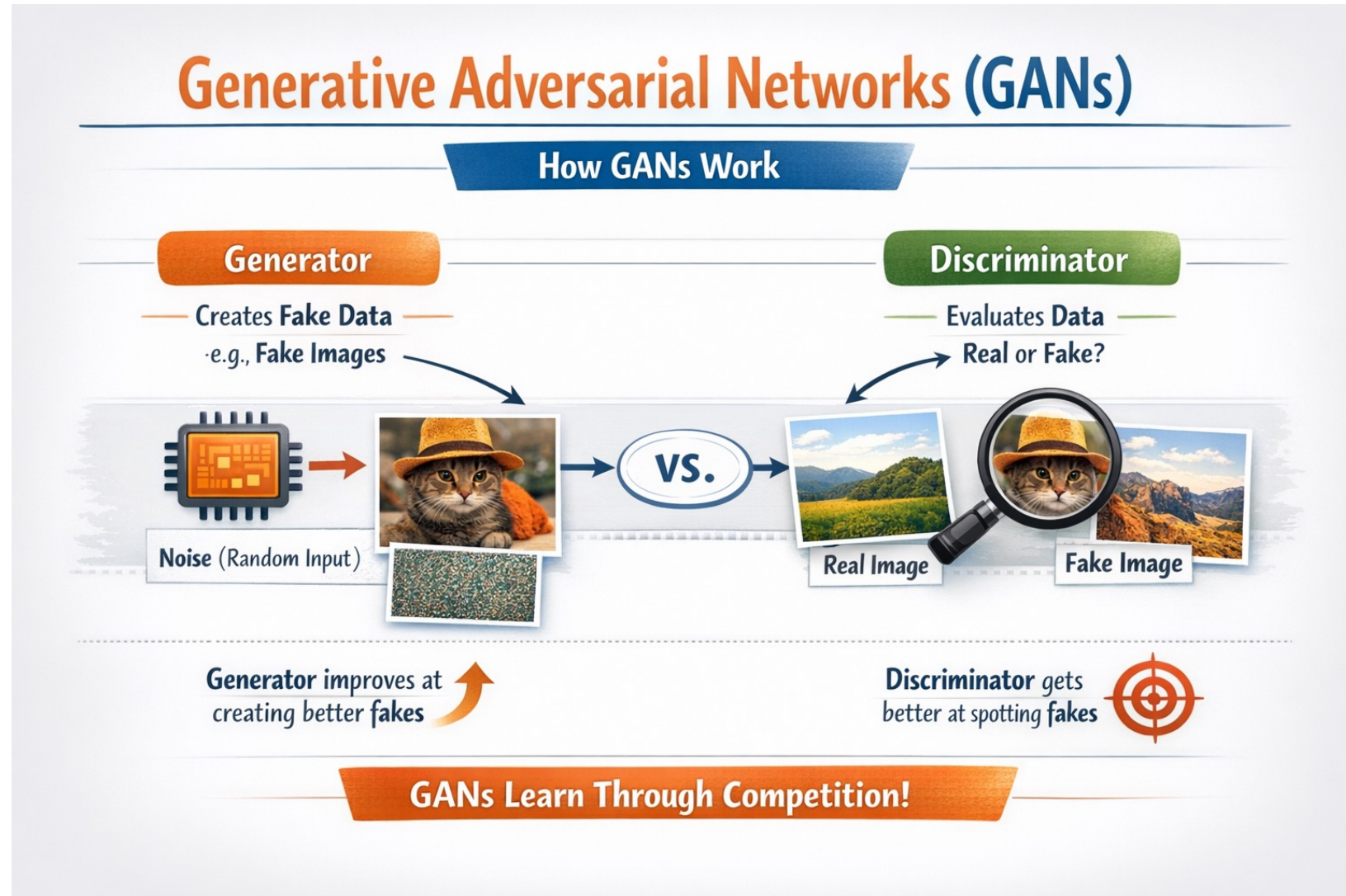
Machine Leaaring course
**Generative Adversarial
(GAwnik)**

Madiineergo ai derieminte

GPT5 (Spring 2026) does a much better (but not detailed!) job at explaining GANs

Prompt: Generate a slide (as an image) explaining Generative Adversarial Networks (GANs) for a Machine Learning course, carefully designed, easy to follow

Negative Prompt: boring, unclear, nontechnical



Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input,
and generates an image

2) the Discriminator

takes in an image classifies whether it is
real (label 1) or fake (label 0)

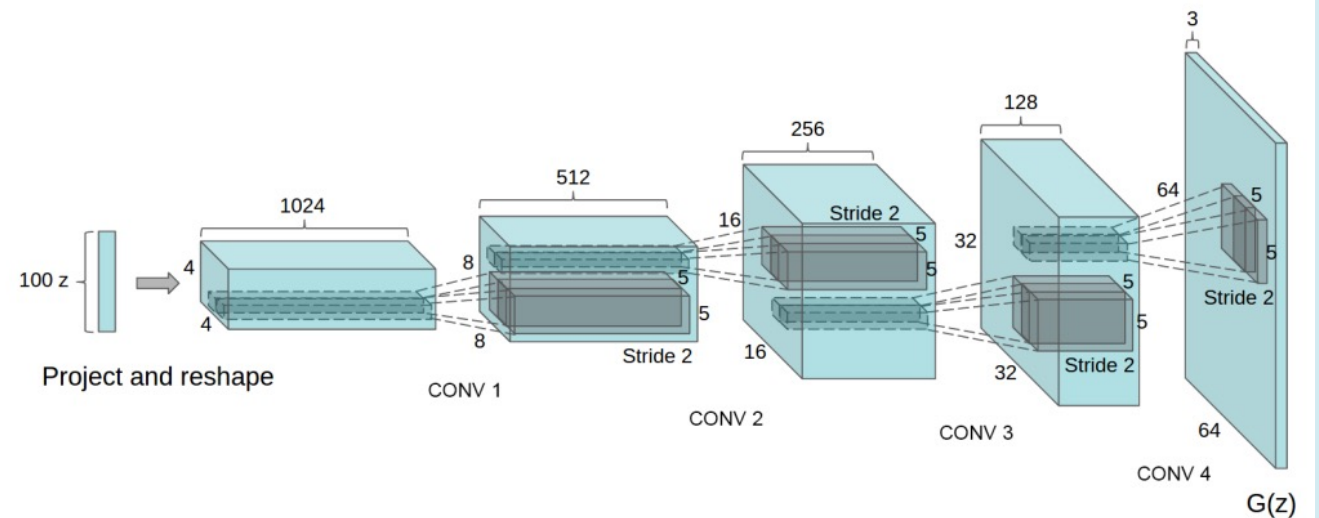
Generator Model

1) the Generator

takes a vector of random noise as input, and generates an image

Example Generator: DCGAN

- An inverted CNN with four **fractionally-strided** convolution layers (not deconvolution)
- These fractional strides grow the size of the image from layer to layer
- The final layer has three channels for red/green/blue



Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input,
and generates an image

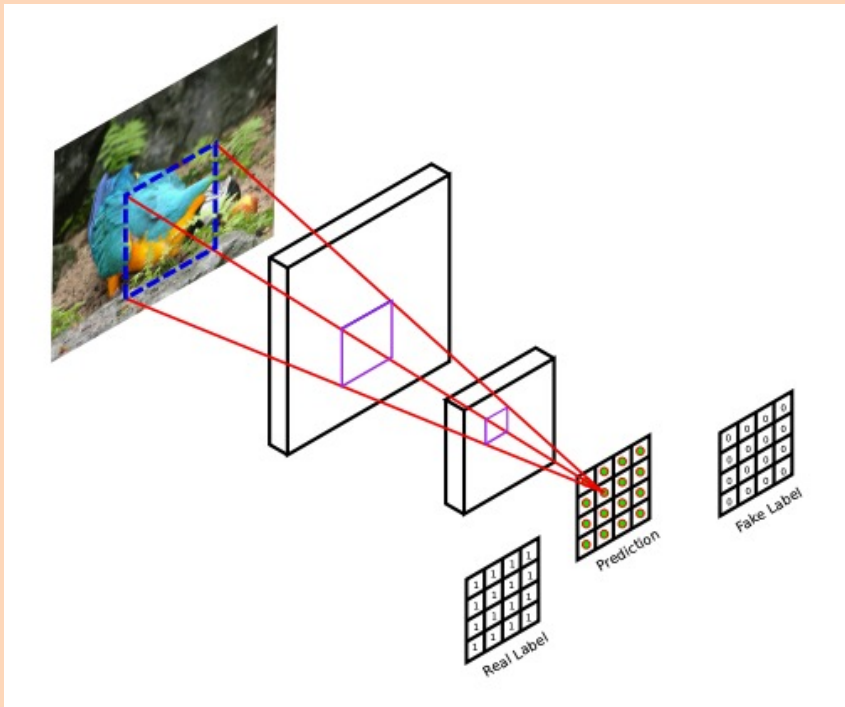
2) the Discriminator

takes in an image classifies whether it is
real (label 1) or fake (label 0)

Discriminator Model

Example Discriminator: PatchGAN

- Convolutional neural network
- Looks at each patch of the image and tries to predict whether it is real or fake
- Helps avoid producing blurry images



2) the Discriminator

takes in an image classifies whether it is real (label 1) or fake (label 0)

Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input, and generates an image

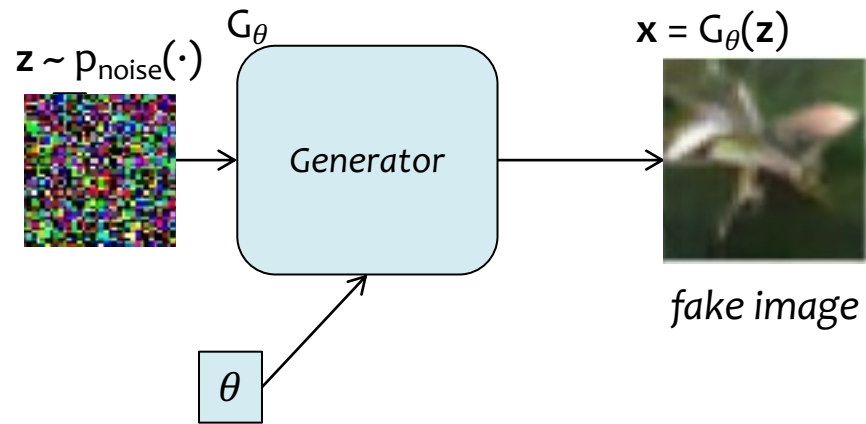
2) the Discriminator

takes in an image classifies whether it is real (label 1) or fake (label 0)

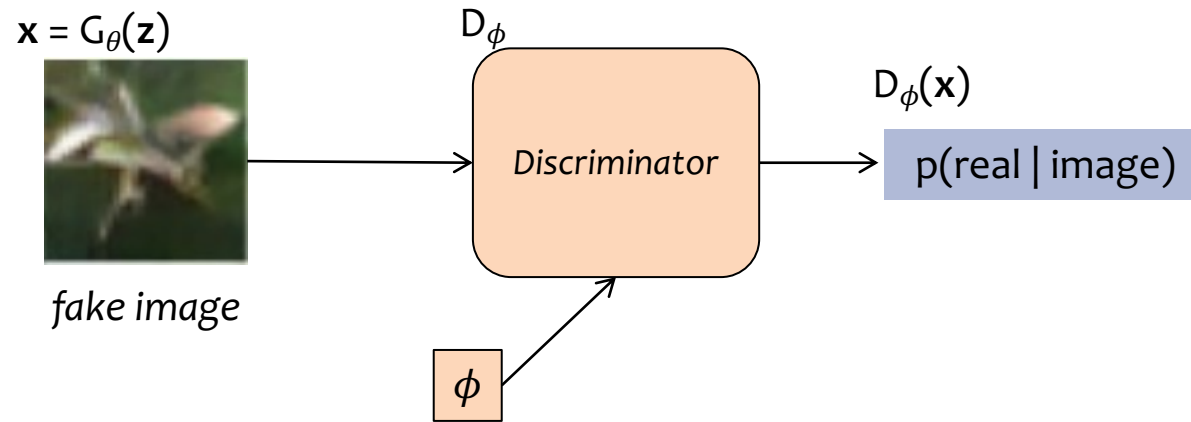
In training, the GAN plays a two player minimax game:

1. the Generator tries to create realistic images to fool the Discriminator into thinking they are real
2. the Discriminator tries to identify the real images from the fake

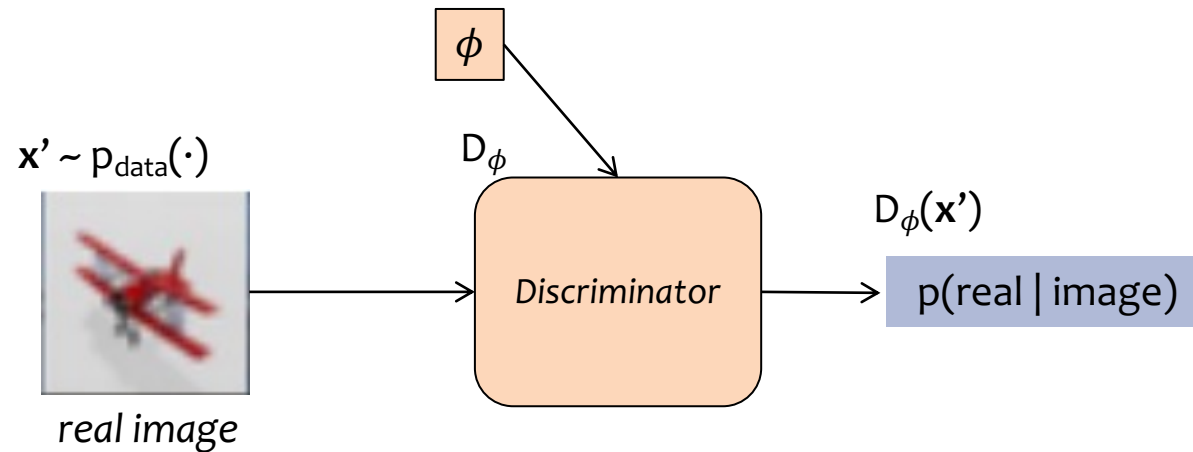
Generative Adversarial Networks (GANs)



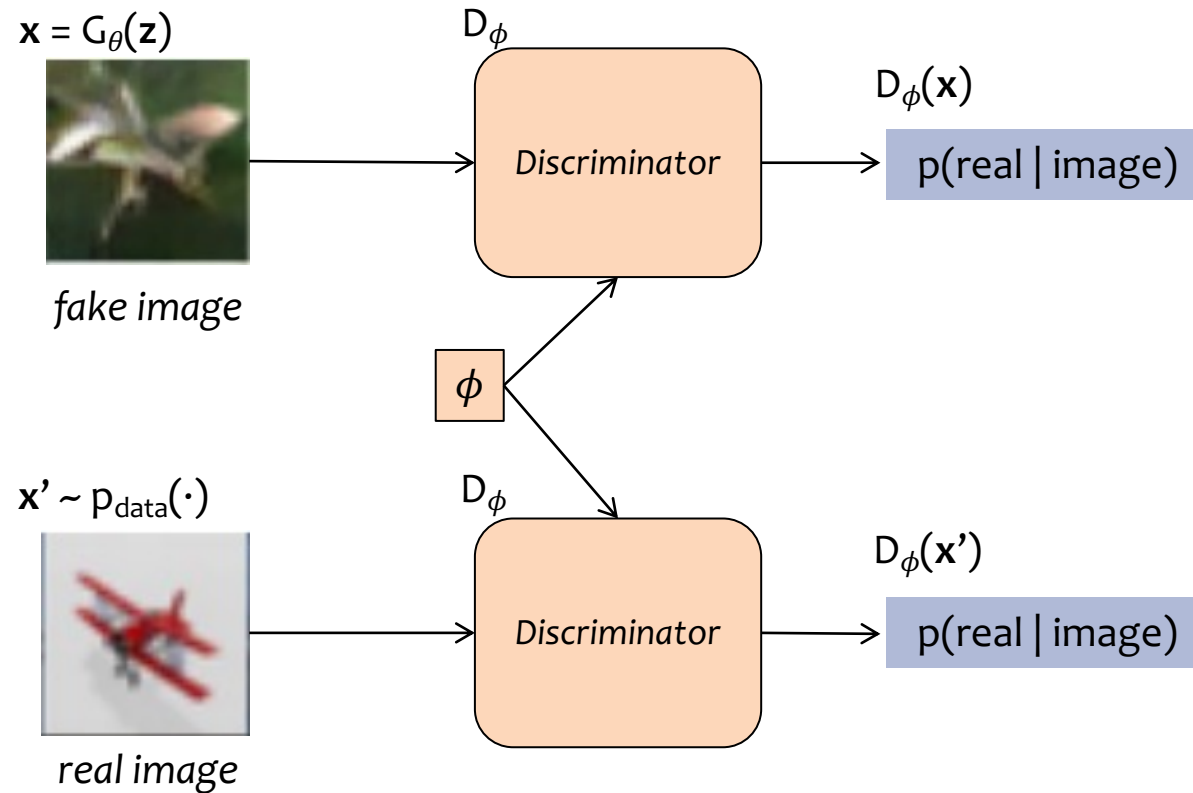
Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)



LEARNING FOR GANS

Generative Adversarial Networks (GANs)

A GAN consists of two deterministic neural network models:

1) the Generator

takes a vector of random noise as input, and generates an image

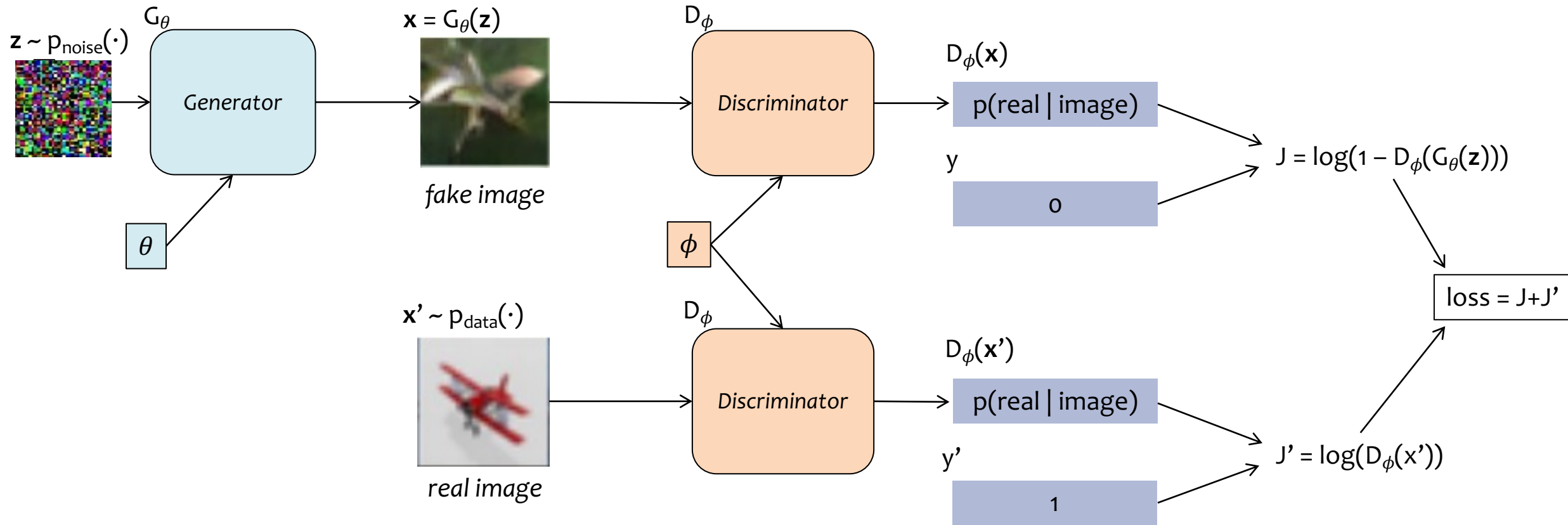
2) the Discriminator

takes in an image classifies whether it is real (label 1) or fake (label 0)

In training, the GAN plays a two player minimax game:

1. the Generator tries to create realistic images to fool the Discriminator into thinking they are real
2. the Discriminator tries to identify the real images from the fake

Generative Adversarial Networks (GANs)



Generative Adversarial Networks (GANs)

$$\max_{\phi} J(\theta, \phi)$$

$$\max_{\phi} \log \left(D_{\phi}(\mathbf{x}^{(i)}) \right) + \log \left(1 - D_{\phi}(G_{\theta}(\mathbf{z}^{(i)})) \right)$$

$$\min_{\theta} \log \left(1 - D_{\phi}(G_{\theta}(\mathbf{z}^{(i)})) \right)$$

$$\min_{\theta} J(\theta, \phi)$$

The discriminator is trying to maximize the likelihood of a binary classifier with labels {real = 1, fake = 0}, on the fixed output of the generator

The generator is trying to minimize the likelihood of its generated (fake) image being classified as fake, according to a fixed discriminator

In training, the GAN plays a two player minimax game:

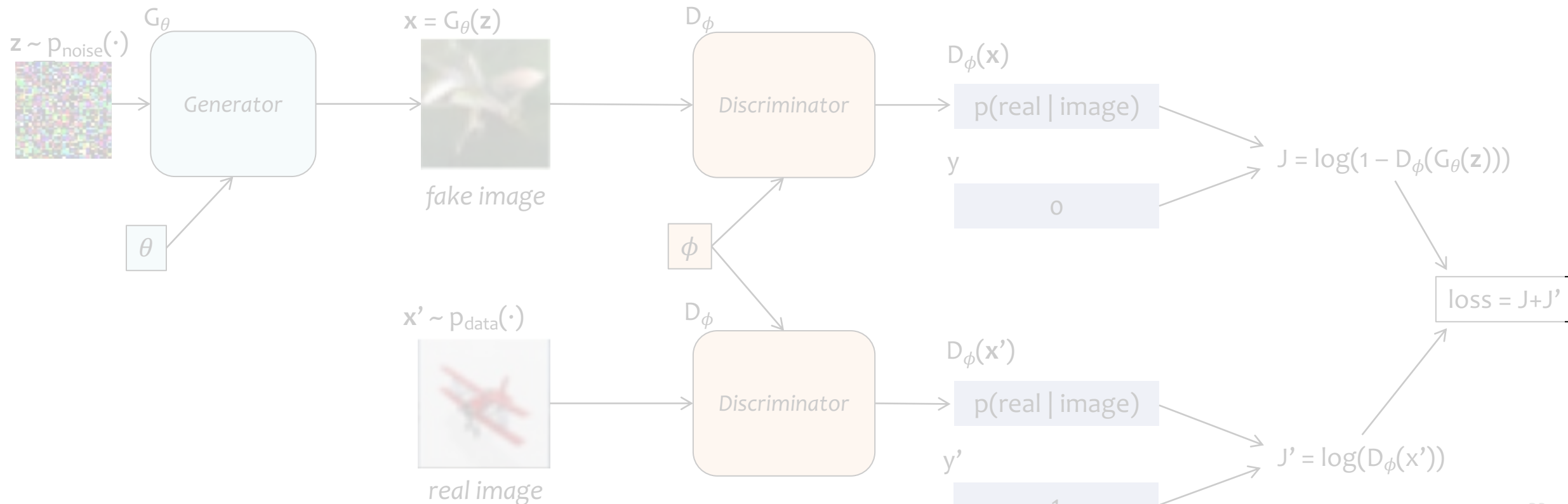
1. the Generator tries to create realistic images to fool the Discriminator into thinking they are real
2. the Discriminator tries to identify the real images from the fake

Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- Keep D_ϕ fixed and backprop through G_θ

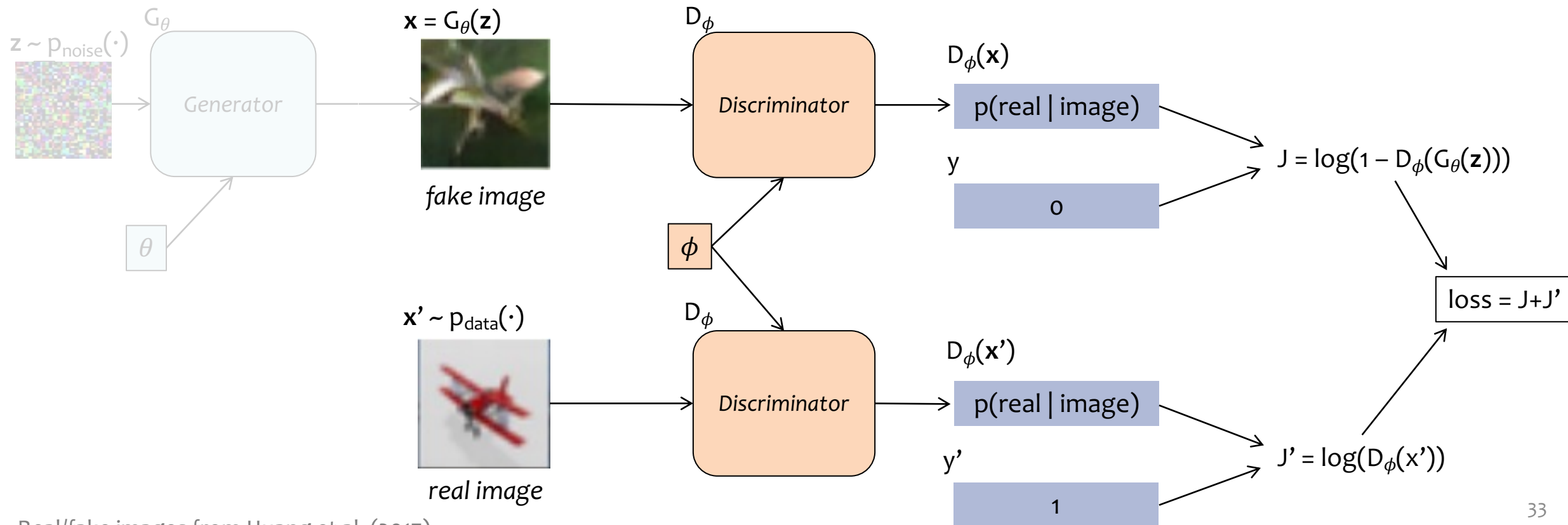


Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- **Keep G_θ fixed and backprop through D_ϕ**
- **Keep D_ϕ fixed and backprop through G_θ**

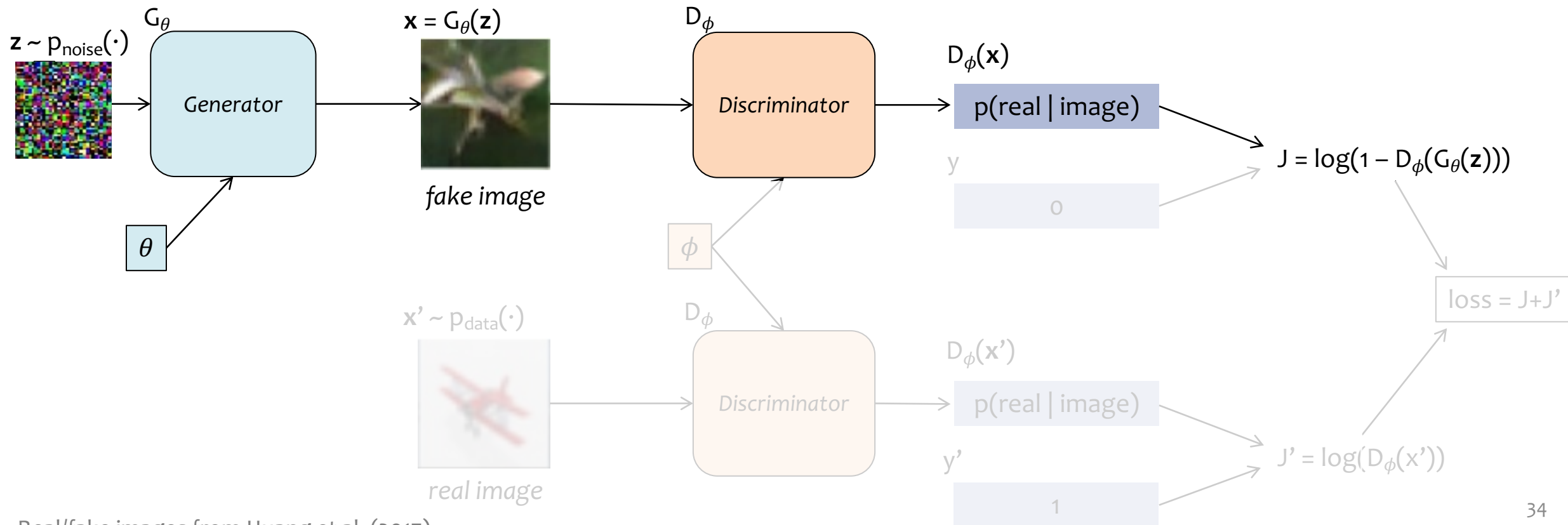


Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- **Keep D_ϕ fixed and backprop through G_θ**

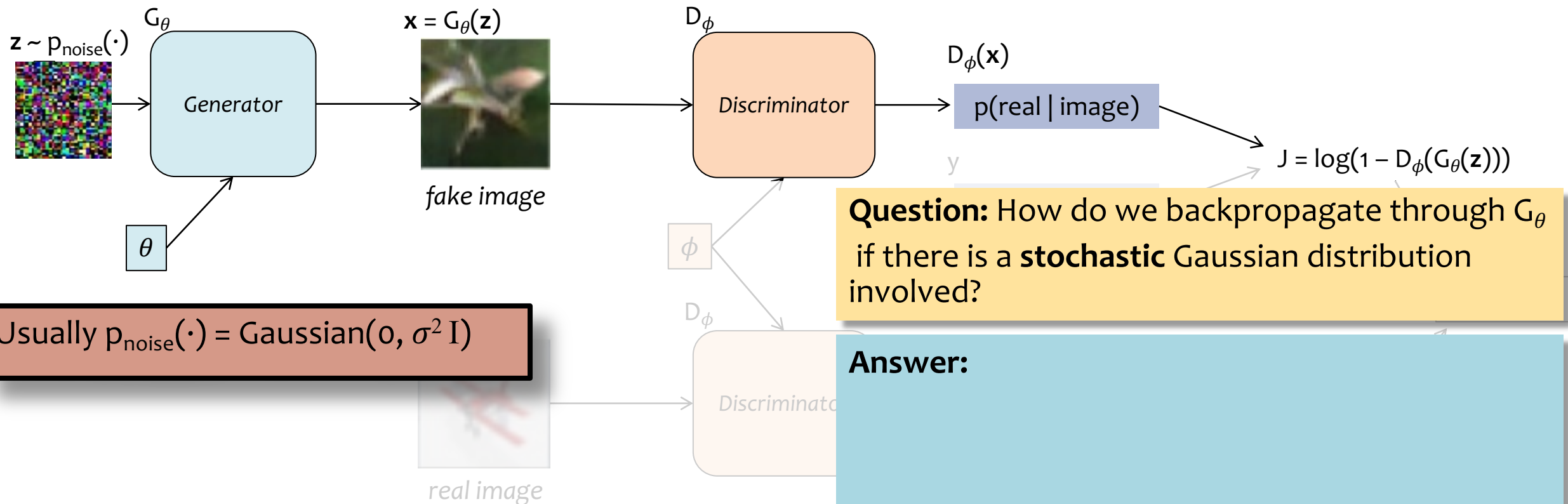


Learning a GAN

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- **Keep D_ϕ fixed and backprop through G_θ**



Usually $p_{\text{noise}}(\cdot) = \text{Gaussian}(0, \sigma^2 I)$

Learning a GAN

- Training data consists of a collection of m unlabeled images $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}$
- Optimization is similar to block coordinate descent
- But instead of exactly solving the min/max problem, we take a step of mini-batch SGD

Algorithm 1 Minibatch stochastic gradient descent training of generative adversarial nets. The number of steps to apply to the discriminator, k , is a hyperparameter. We used $k = 1$, the least expensive option, in our experiments.

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Sample minibatch of m examples $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\mathbf{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^m \left[\log D(\mathbf{x}^{(i)}) + \log \left(1 - D(G(\mathbf{z}^{(i)})) \right) \right].$$

end for

- Sample minibatch of m noise samples $\{\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(m)}\}$ from noise prior $p_g(\mathbf{z})$.
- Update the generator by descending its stochastic gradient:

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^m \log \left(1 - D(G(\mathbf{z}^{(i)})) \right).$$

end for

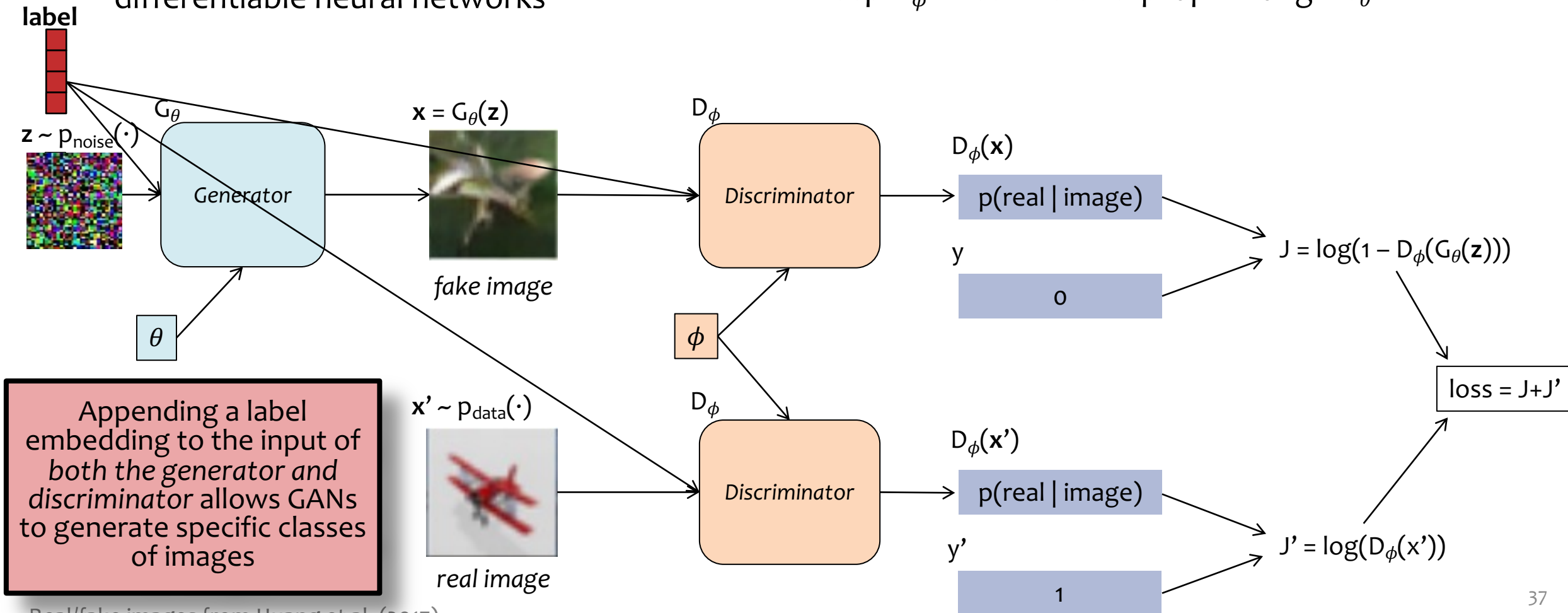
The gradient-based updates can use any standard gradient-based learning rule. We used momentum in our experiments.

Class-conditional GANs

- Objective function is a simple differentiable function
- We chose G and D to be differentiable neural networks

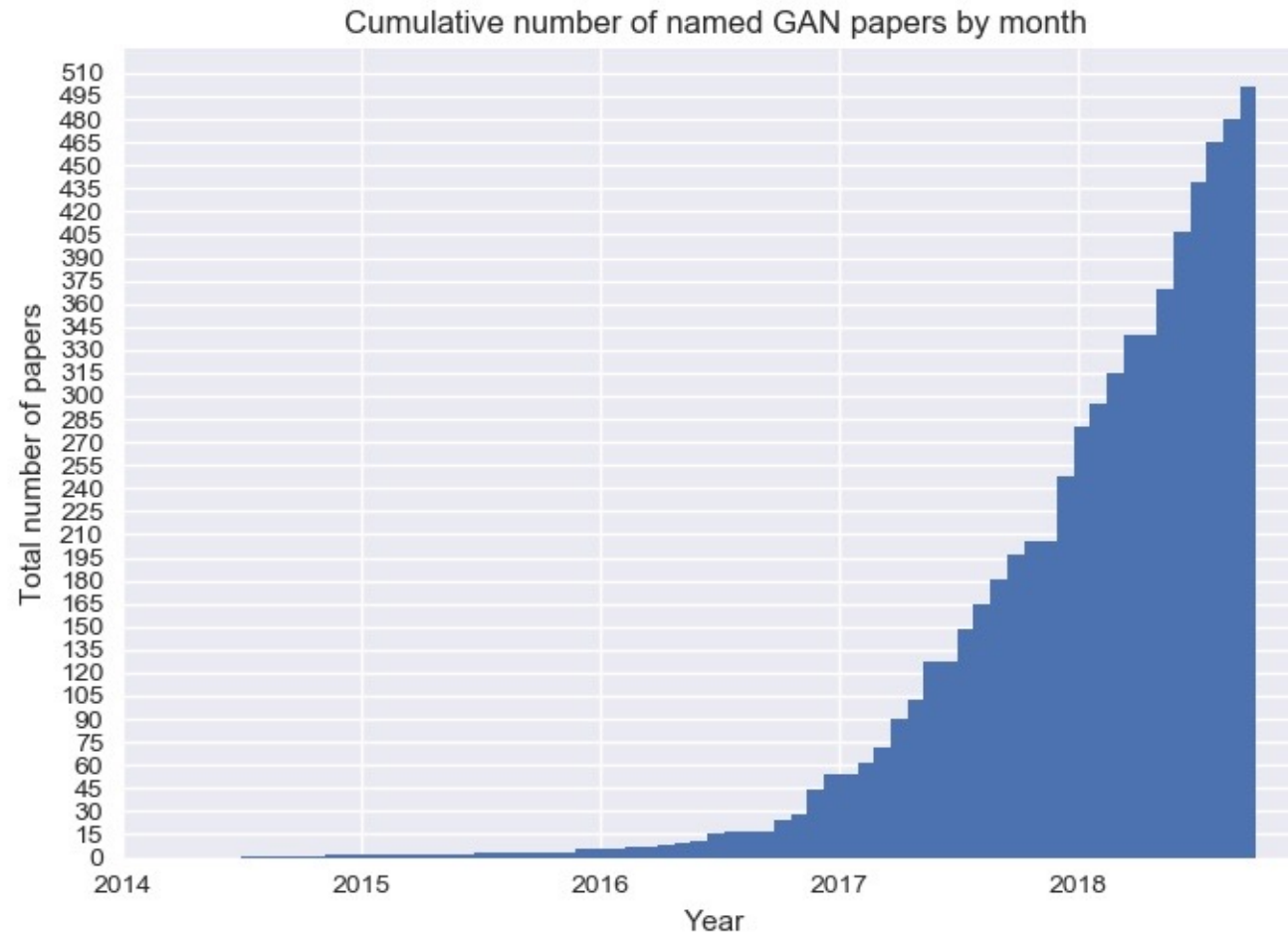
Training alternates between:

- Keep G_θ fixed and backprop through D_ϕ
- Keep D_ϕ fixed and backprop through G_θ



SCALING UP THE MODEL SIZE

GANs Everywhere!



Recall: Computer Vision Timeline

